

WMM – Lab 8

Generowanie grafiki z wykorzystaniem popularnej biblioteki graficznej

Autor: Łukasz Dąbała, Michał Chwesiuk

1 Wstęp

Celem laboratorium 8 jest zapoznanie się z biblioteką graficzną OpenGL oraz podstawami programowania na GPU. W tym celu do zrealizowania będą 3 ćwiczenia poruszające zagadnienia wyświetlania grafiki z jej użyciem.

Przydatne linki:

1. Dokumentacja OpenGL - <https://www.khronos.org/registry/OpenGL-Refpages/gl4/>

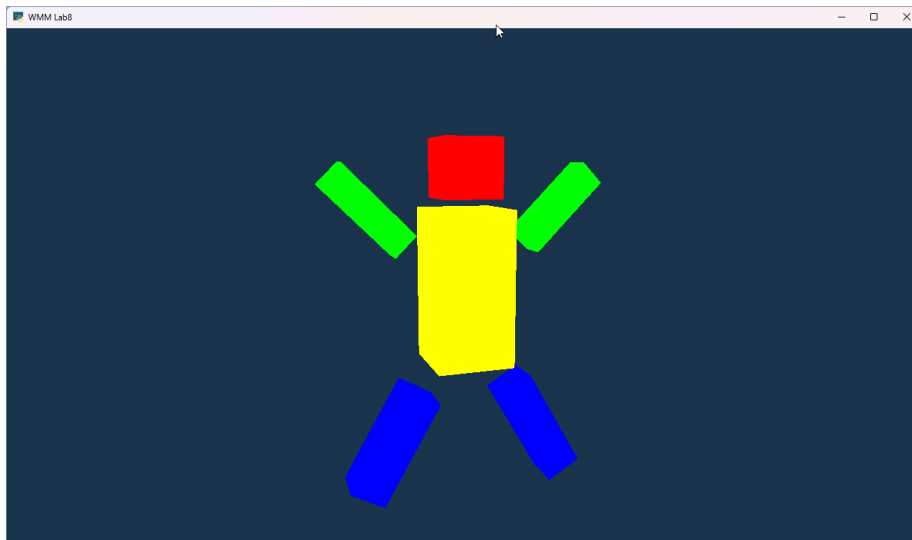
2 Wymagania

W celu realizacji laboratorium niezbędny jest interpreter języka Python w wersji 3.11.3. Aby ułatwić też sobie pracę, preferowanym narzędziem do pisania kodu będzie środowisko programistyczne PyCharm. Ze względu na pracę z shaderami, przydatna może być też wtyczka do tego programu: GLSL Support, która dodaje podstawowe kolorowanie składni.

W plikach dołączonych do zadania, znajduje się kod stanowiący podstawę do modyfikacji w trakcie trwania laboratorium. W środku archiwum znajdują się pliki Pythonowe oraz wzorcowe shadery, które będą mogły stanowić bazę do wykonania ćwiczeń. Sam program umożliwia wczytanie shaderów z plików *.frag* (*fragment shader*) i *.vert* (*vertex shader*) oraz wyświetlenie podstawowego okna. Aby ułatwić instalację niezbędnych pakietów w paczce znajduje się plik *requirements.txt*.

3 Sprawozdanie i kod

Kod tworzony w ramach laboratorium powinien być napisany w sposób zrozumiały oraz powinien być skomentowany. Komentarze w kodzie powinny tłumaczyć co dzieje się w danym miejscu np. *obliczanie wektora z danego wierzchołka w kierunku źródła światła*. Do opisu działania można wykorzystać zarówno język polski i angielski.



Rysunek 1: Przykładowe złożenie transformacji dla obiektów w cel ułożenia z nich robota.

Dodatkowo należy stworzyć sprawozdanie, które będzie dokumentować działanie stworzonych programów. W opisie każdego zadania znajdują się również elementy, które powinny znaleźć się w sprawozdaniu.

4 Zadanie 1: Składanie transformacji (2.0 pkt)

Celem tego zadania jest złożenie transformacji obiektów w scenie 3D. Efektem końcowym wykonanego zadania powinna być scena zawierająca robota złożonego z trójwymiarowych sześcianów, tak jak na rysunku 1. Zadanie powinno zostać wykonane w klasie *RobotWindow*, która znajduje się w pliku *robot_window.py*.

Pierwszym etapem zadania jest wygenerowanie buforu pamięci znajdującego się w pamięci GPU (Vertex Array Object - *VAO*), który będzie zawierał dane geometrii przedstawianego sześcianu. Funkcja, która realizuje ten etap jest zawarta w dostarczonym kodzie, i znajduje się ona w pliku *models.py* pod nazwą *load_cube()*.

Kolejnym etapem jest napisanie programu cieniującego (ang. *shader program*). Programy cieniujące zaimplementowane do realizacji zadania powinny znaleźć się w folderze *resources/shaders/robot*, jako pliki *robot.vert* i *robot.frag*. W odróżnieniu od shader'ów zawartych w dostarczonym projekcie (*passthrough.vert* i *passthrough.frag*), które wyłącznie przenosiły pozycję dwuwymiarowych wierzchołków do kolejnych etapów renderingu. Shader'y potrzebne do wykonania tego zadania będą miały za zadanie przekształcenie trójwymiarowych wierzchołków do dwuwymiarowej przestrzeni obrazu wyświetlanego na ekranie. Tą operację realizuje się za pomocą formuły:

$$gl_Position = P * V * M * vert_{in}$$

P , V , M oznaczają kolejno macierz projekcji, widoku i modelu o rozmiarze 4x4, które w *GLSL* są typu *mat4*. $vert_{in}$ oznacza wejściowy trójwymiarowy wierzchołek w postaci jednorodnej, a $gl_Position$ oznacza wierzchołek przekształcony do przestrzeni obrazu przekazywany do kolejnych etapów renderingu.

Następnie, należy skonstruować macierze, które będziemy przekazywać do programu cieniującego. W tym celu należy wykorzystać pakiet *pyrr*. Przydatne będą następujące metody:

1. `Matrix44.perspective_projection`

- macierz projekcji perspektywicznej

2. `Matrix44.look_at`

- macierz w celu skierowania wzroku na dany punkt

Macierz modelu należy skonstruować z pod-macierzy realizujących poszczególne przekształcenia geometryczne:

1. `Matrix44.from_translation`

- macierz translacji tworzona z wektora

2. `Matrix44.from_scale`

- macierz skalowania tworzona z wektora skali dla poszczególnych osi

3. `Matrix44.from_eulers`

- macierz obrotu tworzona z podanych kątów Eulera

4. `Matrix44.from_x_rotation`

`Matrix44.from_y_rotation`

`Matrix44.from_z_rotation`

- macierze obrotu wokół odpowiednich osi tworzona z podanej rotacji

Generowanie macierzy powinno następować za każdym razem, gdy chcemy generować ramkę obrazu (w funkcji *render()*).

Wygenerowane macierze w kodzie Python należy przekazać do shader'a. W tym celu zadeklarować zmienne w kodzie shader'a ze słowem kluczowym *uniform*. Następnie, należy pobrać lokacje (identyfikator) tych zmiennych w przygotowanej funkcji *init_shaders_variables()*. Przykład:

```
self.uniform_location = self.program['variable_name']
```

Mając lokacje, możemy przekazać wartości do zmiennej *uniform* w programie cieniującym:

```
self.uniform_location.write(vector_variable)
self.uniform_location.write(matrix_variable.astype('f4'))
```

Po implementacji przekształceń geometrycznych, możemy wygenerować kilka sześcianów o różnych pozycjach, rozmiarach, i rotacjach, w taki sposób, by przypominały robota. Przykładowa konfiguracja: Podstawowe kształty, które dostarczone są w ramach zasobów do laboratorium należy poddać modyfikacjom:

1. głowa - translacja o wektor $t = (0, 5, 0)$, skalowanie we wszystkich osiach ze współczynnikiem 1,5.
2. ciało - translacja o wektor $t = (0, 2, 0)$, skalowanie w osiach XYZ o wektor $s = (2, 4, 2)$
3. ręce - translacja o wektor $t = (-/+2.5, 4, 0)$, rotacja o kąt $-/+45^\circ$ wokół osi z , skalowanie ze współczynnikami $s = (0.75, 2.5, 0.75)$
4. nogi - translacja o wektor $t = (-/+2, -2, 0)$, rotacja o kąt $+/-30^\circ$ wokół osi z , skalowanie ze współczynnikami $s = (1, 3, 1)$

Jak można zauważyć na rysunku 1, obiekty są również pokolorowane na różne sposoby, w związku z tym należy zmodyfikować shader'y.

1. **vertex shader** - przekształcenie geometryczne
2. **fragment shader** - dodanie koloru, który jest identyczny dla wszystkich fragmentów (*uniform vec3*)

Dodatkowym wymaganiem zadania jest spersonalizowanie wyglądu robota na podstawie numeru indeksu studenta. W tym celu należy w programie zdefiniować zmienną przechowującą numer indeksu, a następnie wykorzystać operację modulo do wyboru koloru głowy oraz dodatkowej bryły.

Kolor głowy powinien zależeć od wartości:

$$color_{id} = nr_{indeksu} \mod 8$$

Należy wykorzystać następujące kolory:

1. $color_{id} = 0$ – czerwony, RGB: (1.0, 0.0, 0.0),
2. $color_{id} = 1$ – pomarańczowy, RGB: (1.0, 0.5, 0.0),
3. $color_{id} = 2$ – żółty, RGB: (1.0, 1.0, 0.0),
4. $color_{id} = 3$ – zielony, RGB: (0.0, 0.8, 0.2),
5. $color_{id} = 4$ – cyjanowy, RGB: (0.0, 0.8, 1.0),

6. $color_{id} = 5$ – niebieski, RGB: (0.0, 0.2, 1.0),
7. $color_{id} = 6$ – fioletowy, RGB: (0.6, 0.0, 1.0),
8. $color_{id} = 7$ – różowy, RGB: (1.0, 0.0, 0.6).

Dodatkowo, robot powinien zostać rozszerzony o jedną bryłę, zależną od numeru indeksu studenta. W tym celu należy obliczyć wartość:

$$element_{id} = nr_{indeksu} \mod 3$$

Należy wykorzystać następujące warianty:

1. $element_{id} = 0$ – czapka w kształcie ostrosłupa umieszczona na głowie robota,
2. $element_{id} = 1$ – kulka przyczepiona do ręki robota,
3. $element_{id} = 2$ – torus umieszczony na wysokości pasa, przypominający koło ratunkowe.

Przykładowo, dla numeru indeksu 12345 należy obliczyć:

$$color_{id} = 12345 \mod 8 = 1$$

$$element_{id} = 12345 \mod 3 = 0$$

czyli kolor pomarańczowy głowy oraz ostrosłup przedstawiająca czapkę.

Na tej podstawie należy wybrać kolor głowy z tabeli kolorów oraz odpowiednią bryłę.

W sprawozdaniu należy podać użyty numer indeksu, obliczone wartości $color_{id}$ oraz $element_{id}$. Należy także zawrzeć wizualizację stworzonego robota w innych kolorach lub przekształceniu niż ten pokazany w instrukcji.

5 Zadanie 2: Animacja geometrii (1.0 pkt)

Celem tego zadania jest dodanie prostej animacji do robota zbudowanego w poprzednim zadaniu. Animacja powinna polegać na harmonicznym poruszaniu wybraną kończyną robota, np. machaniu ręką, poruszaniu nogą lub obracaniu głowy.

Zadanie powinno zostać wykonane poprzez modyfikację macierzy modelu wybranego elementu robota w każdej klatce renderowania. W tym celu należy wykorzystać czas działania programu i na jego podstawie obliczyć zmienny kąt obrotu.

Animacja powinna mieć charakter harmoniczny, tzn. wartość kąta obrotu powinna być obliczana z użyciem funkcji trygonometrycznych, np:

$$angle = A \cdot \sin(\phi \cdot time)$$

gdzie:

1. A – amplituda ruchu, czyli maksymalne wychylenie kończyny,
2. ϕ – szybkość animacji,
3. $time$ – czas działania programu.

Przykładowo, dla animacji machania ręką można przyjąć:

$$A = 45^\circ$$

$$\phi = 2.0$$

Oznacza to, że ręka robota będzie płynnie obracać się w zakresie od -45° do 45° .

Animacja powinna zostać obliczona po stronie CPU. Następnie, na podstawie obliczonego kąta, należy zbudować macierz modelu animowanej kończyny i przesłać ją do programu cieniującego jako zmienną *uniform*.

Przykładowy sposób obliczenia kąta animacji po stronie CPU:

$$angle = A \cdot \sin(\phi \cdot time)$$

Następnie kąt ten powinien zostać wykorzystany przy budowie macierzy obrotu wybranego elementu robota.

W przypadku animacji ręki należy zwrócić uwagę na punkt obrotu. Ręka powinna obracać się w pobliżu ramienia, a nie wokół własnego środka. Można to osiągnąć przez odpowiednie złożenie transformacji: translacji do punktu zaczepienia kończyny, obrotu, przesunięcia lokalnego oraz skalowania.

Przykładowy schemat transformacji dla animowanej ręki:

$$M = T_{shoulder} \cdot R_z(angle) \cdot T_{local} \cdot S$$

gdzie:

1. $T_{shoulder}$ – translacja do punktu zaczepienia ręki,
2. $R_z(angle)$ – obrót wokół osi z o kąt zależny od czasu,
3. T_{local} – lokalne przesunięcie geometrii ręki względem punktu obrotu,
4. S – skalowanie ręki.

Student powinien zaimplementować animację co najmniej jednej kończyny robota. Dodatkowo można rozszerzyć animację o drugi element, np. przeciwną rękę poruszającą się w przeciwfazie.

Przykładowo, dla dwóch rąk można użyć zależności:

$$left_{angle} = A \cdot \sin(\omega \cdot time)$$

$$right_{angle} = -A \cdot \sin(\omega \cdot time)$$

W sprawozdaniu należy zawrzeć:

1. opis wybranej kończyny poddanej animacji,
2. wzór użyty do obliczenia kąta animacji,
3. wartości parametrów A oraz ω ,
4. zrzuty ekranu pokazujące robota w co najmniej dwóch różnych fazach animacji,
5. krótki opis sposobu złożenia transformacji animowanego elementu.

6 Zadanie 3: Model Oświetlenia Phong (2.0 pkt)

Celem tego zadania jest implementacja oraz porównanie dwóch lokalnych modeli oświetlenia: modelu Phong oraz modelu Blinna-Phonga. W instrukcji podany jest sposób implementacji składowych ambient oraz diffuse. Należy rozbudować go o składową specular, a następnie przygotować dwa warianty obliczania tej składowej: klasyczny model Phong oraz model Blinna-Phonga.

Zadanie powinno zostać wykonane w klasie *PhongWindow*, która znajduje się w pliku *phong.window.py*. Programy cieniujące powinny znaleźć się w katalogu *resources/shaders/phong* w plikach *phong.vert* i *phong.frag*, oraz w katalogu *resources/shaders/blinn_phong* w plikach *blinn_phong.vert* i *blinn_phong.frag*.

Większość obliczeń oświetlenia będzie wykonywana w fragment shaderze. Część danych, tj. pozycja wierzchołka oraz wektor normalny, znajduje się w vertex shaderze, dlatego w pierwszej kolejności należy przekazać te dane do fragment shadera.

W vertex shaderze należy zadeklarować zmienne wyjściowe dla pozycji wierzchołka oraz wektora normalnego:

```
out vec3 v_position;
out vec3 v_normal;
```

W fragment shaderze należy zadeklarować odpowiadające im zmienne wejściowe:

```
in vec3 v_position;
in vec3 v_normal;
```

Pozycję wierzchołka w przestrzeni świata można obliczyć w vertex shaderze w następujący sposób:

$$v_position = (M * vec4(in_position, 1.0)).xyz;$$

Wektor normalny również należy przekształcić do przestrzeni świata. W najprostszy wariant można wykorzystać macierz modelu:

$$v_normal = (M * vec4(in_normal, 0.0)).xyz;$$

W przypadku obiektów poddanych nierównomiernemu skalowaniu poprawniejszym rozwiązaniem jest wykorzystanie macierzy normalnych:

$$v_normal = mat3(transpose(inverse(M))) * in_normal;$$

W fragmencie shadera należy zadeklarować zmienne przechowujące parametry światła, parametry materiału oraz pozycję kamery. Parametry materiału powinny być przekazywane do programu cieniującego za pomocą zmiennych *uniform*.

```
uniform vec3 light_position;
uniform vec3 light_ambient;
uniform vec3 light_diffuse;
uniform vec3 light_specular;
uniform vec3 material_ambient;
uniform vec3 material_diffuse;
uniform vec3 material_specular;
uniform float material_shininess;
uniform vec3 camera_position;
```

Należy upewnić się, że pozycja kamery przekazywana do programu cieniującego jest zgodna z wektorem pozycji kamery wykorzystanej do generacji macierzy widoku, tj. z parametrem *eye* w funkcji *Matrix44.look_at()*.

Składową ambient należy obliczyć za pomocą poniższej formuły:

$$vec3\ ambient = light_ambient * material_ambient;$$

Do obliczenia składowej diffuse należy wyznaczyć znormalizowany wektor normalny N oraz znormalizowany wektor od powierzchni do źródła światła L :

$$vec3\ N = normalize(v_normal);$$

$$vec3\ L = normalize(light_position - v_position);$$

$$float\ cosNL = max(dot(N, L), 0.0);$$

Składową diffuse należy obliczyć za pomocą formuły:

$$vec3\ diffuse = light_diffuse * material_diffuse * cosNL;$$

Następnie należy zaimplementować składową specular w dwóch wariantach. Końcowy kolor fragmentu należy obliczyć jako sumę trzech składowych:

$$vec3\ final_color = clamp(ambient + diffuse + specular, 0.0, 1.0);$$

$$f_color = vec4(final_color, 1.0);$$

Dodatkowym wymaganiem jest wprowadzenie dynamiki oświetlenia. Oznacza to, że co najmniej jeden parametr światła powinien zmieniać się w czasie. Parametr ten powinien być obliczany po stronie CPU, a następnie przekazywany do programu cieniującego za pomocą zmiennej *uniform*.

Przykładowe warianty dynamiki oświetlenia:

1. zmiana pozycji źródła światła w czasie, np. ruch światła po okręgu wokół obiektu,
2. zmiana koloru światła w czasie,
3. zmiana intensywności światła w czasie.

Przykładowo, pozycja światła może być modyfikowana po stronie CPU w każdej klatce renderowania:

$$light_x = r \cdot \sin(time)$$

$$light_y = const$$

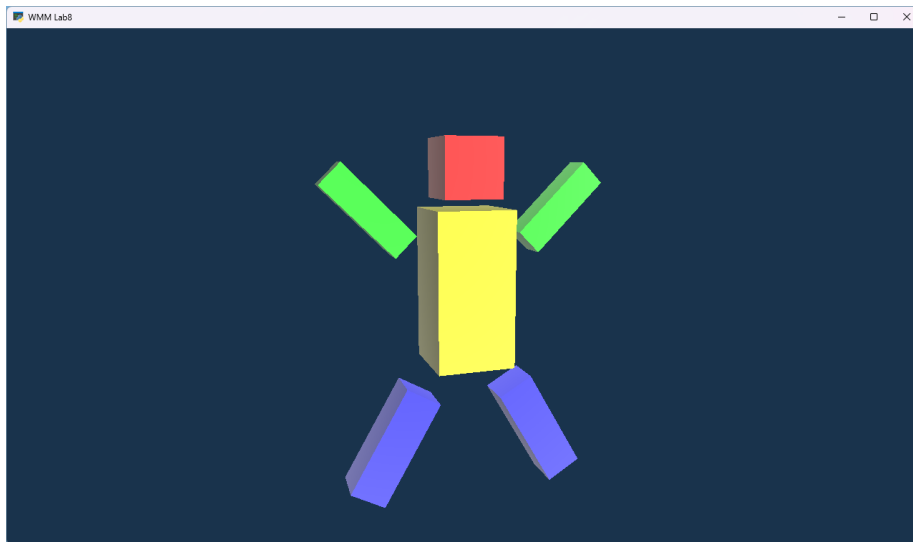
$$light_z = r \cdot \cos(time)$$

$$light_position = (light_x, light_y, light_z)$$

Następnie tak obliczona pozycja światła powinna zostać przesłana do zmiennej *uniform light_position* w programie cieniującym.

GLSL oferuje szereg funkcji, które mogą być przydatne przy implementacji zadania:

1. **transpose(M)**
 - transpozycja macierzy,
2. **inverse(M)**
 - odwrotność macierzy,
3. **normalize(V)**
 - normalizacja wektora,
4. **reflect(I, N)**
 - obliczenie wektora odbitego od powierzchni,
5. **dot(V1, V2)**
 - iloczyn skalarny wektorów,
6. **pow(a, b)**



Rysunek 2: Przykład renderu robota z zaaplikowanym oświetleniem, dla parametru połyskliwości (*shininess*) równej 50.

– podniesienie liczby a do potęgi b ,

7. `max(a, b)`

– wartość maksymalna z dwóch wartości,

8. `clamp(v, a, b)`

– ograniczenie wartości do podanego przedziału.

W sprawozdaniu należy zawrzeć:

1. wizualizację sceny dla modelu Phong'a,
2. wizualizację sceny dla modelu Blinna-Phong'a,
3. krótkie porównanie uzyskanych efektów wizualnych, szczególnie w obszarze refleksów światła,
4. opis zastosowanej dynamiki oświetlenia,
5. co najmniej jedną wizualizację pokazującą scenę dla zmienionego położenia, koloru lub intensywności światła.

Przykładowa wizualizacja znajduje się na rysunku [2](#).